

AGENTIC OPERATOR · 发明专利交底

发明主线 · 专利交底书

从真实代码库挖掘 · 全程白话填写 · 2026-06-12

壹 一种将内部 AI 智能体以"影子模式"安全对外开放为可调用执行服务的方法及系统

贰 一种运行记录就地随手存、系统宕机也不丢的可观测性容灾方法及系统

叁 一种用多家 AI 组成"陪审团"持续核查 AI 业务结论是否有依据的方法及系统

壹 解决什么技术问题 *

我们的 AI 审核 agent 一旦运行,就会真写数据库、真发通知、真推进招聘流程。直接让外部来调用测试,会污染真实数据、误启动真实业务,还可能因为反复请求而重复花掉大模型的钱(每次推理都很贵)。需要一种让外部能安全试用、不动一条真实业务、不重复烧钱、还能核对所用规则版本的办法。

贰 用什么技术手段 *

- 空跑演练(影子模式)。照常算出结论,但所有"本该写库、发通知、推进流程"的动作,只在结果里记一笔"本该干啥",一件都不真做。
- 同一请求只算一次。每个请求带个编号,重复发来直接返回上次结果;就算两个一样的请求同时挤进来,数据库这一层也卡死只让一个真算,不重复花大模型的钱。
- 太忙就请你稍后再来。系统盯着"手头正在算几单",超过上限就回一句"请 5 秒后重试",保护后端大模型不被外部并发压垮。
- 结果可核对版本。每条用到的规则都带上它原文的"指纹"随结论返回;对方点名的版本我们没有就直接报错,绝不偷偷换成最新版。
- 痕迹单独存放。外部的试用记录只往一张专门的表里写、打上"对外试用"标签,不混进自家的错误率、成本、监控里。

叁 达到什么技术效果 *

- ✓ 外部放心试用、我们零风险:不写一条真实数据、不误触发一个真实流程。
- ✓ 同一请求不重复跑大模型,省钱;后端大模型不会被外部并发打垮。
- ✓ 测试结果可复现,当时用了哪一版规则、原文是什么都能核对。
- ✓ 机器故障(超时、连不上、被限流)绝不会被当成"这个人没通过"的业务结论返回。

建议标题 （可选 · 供命名参考）

一种将内部 AI 智能体以"影子模式"安全对外开放为可调用执行服务的方法及系统

补充说明 （可选）

应用场景。把已上线的招聘规则审核 AI,开放给外部合作伙伴做回归测试与试评估,要求外部能安全调用、不污染我们的真实业务。

与常规做法的差异。一般人想把自家AI对外开放给人试用,通常会另起一套假环境、做个"沙箱"让外面去空跑,等于要养两套东西,还容易跟真的那套跑偏、对不上。我们不另搭环境:外面跑的还是同一套真AI,只是把"本该写真实数据、本该发通知、本该往下推流程"这几件事,改成"只在结果里写一句'这一步本来要干啥',但就是不真干"。再叠上几招——同一个请求只算一次不重复烧钱、后台太忙就回一句"5秒后再来"护住贵的大模型、结果里带着"用的哪版规则的原文指纹"能让对方核对、外面留下的痕迹单独存放不混进自家监控。这几招是咬在一起的一整组,别人想照抄就得把整组都复制下来,单抄一两条没用,所以不好绕。

名词解释。

影子模式(空跑演练):让外面把我们的AI当真跑一遍,结论照常算,但所有会动真实数据、发通知、推流程的动作都只'记一笔本该干啥'而不真干,像彩排不上台。

同一请求只算一次:每个请求带个自己的编号,算过就直接还上次结果,绝不让贵的AI因为网络重发而白跑两遍。

软刹车 / 稍后重试:系统盯着手头正在算的有几单,太多了就回一句'5秒后再来',让外面等一等,免得把后端贵的AI压垮。

后端大模型:真正干推理活的那个AI,算一次又慢又费钱,所以前面要又是去重又是限流地护着它。

本该干啥清单(被抑制的动作):针对这次结论,把真实流程里本来要做的事(写主表、发消息、发通知)一条条列出来放进结果里,看得见但一件都不真做。

壹 解决什么技术问题 *

我们用一个"运行引擎"来跑各种 AI 任务,它像现场记录员,但记录是临时的——一重启或崩溃,还没存档的就没了。常规做法是另开个程序每 30 秒去抄一次,可 30 秒内就跑完、又恰好赶上崩溃的任务,记录会永久丢失。要么把存账塞进正事里同步做,可数据库一卡,正事就被拖死。

贰 用什么技术手段 *

- 事情一发生就顺手存。任务一开始 / 一结束 / 一出错,就在引擎干活的当口直接写进数据库,不用等那固定周期的抄写,任务一结束账就落地。
- 存档出错绝不连累正事。存档万一失败只在旁边记一句告警,正在跑的业务任务照常完成,存档和正事彻底隔开。
- 状态只进不退。已经"完成 / 失败"的记录,绝不会被一条慢半拍、迟到的"进行中"消息打回"还在跑"。
- 看板双保险。看监控时同时问"实时引擎"和"数据库存档"两边:都在以实时为准、用存档补历史;引擎挂了自动只看存档,恢复了又自动切回实时。

叁 达到什么技术效果 *

- ✓ 监控数据几乎不丢:丢失窗口从约 30 秒压到接近 0。
- ✓ 引擎宕机 / 重启时,监控看板照样能看;引擎恢复后自动回到实时。
- ✓ 存档数据库出故障,也不影响业务任务正常跑完。
- ✓ 短平快的小任务(30 秒内起止)不再被定时抄写漏掉。

建议标题 (可选 · 供命名参考)

一种运行记录就地随手存、系统宕机也不丢的可观测性容灾方法及系统

补充说明（可选）

应用场景。以"运行引擎"为核心、任务短平快又可能频繁重启的多 AI 控制平面,需要监控数据在引擎崩溃时也不丢、看板照常能看。

与常规做法的差异。老办法是另派一个人每隔半分钟去运行引擎那儿抄一遍账,本质是"事后旁观",所以快进快出的小任务整个发生在两次抄账的空当里,根本抄不到;引擎一崩,这半分钟的活儿就没了。也有人干脆把"存账"塞进正事里同步做,可一旦数据库卡住,正事就被拖死。本方案的新在于:把存账动作直接挂进引擎跑活儿的几个关键节点上,在同一个进程里就地随手存,任务刚结束那一刻账就落地了,不用等下一次抄。同时立了几条死规矩:存账出错只记一句告警绝不往正事上甩;已经"完成/失败"的状态绝不会被一条迟到的"进行中"覆盖回去。看监控时还同时问实时引擎和数据库两边,谁活着信谁。别人想绕,得把"就地随手存+出错不连累正事+状态不回退+两边对账"这一整套都凑齐,单抄一条没用。

名词解释。

运行引擎:负责真正跑各种自动任务的那台'发动机',它脑子里记着每个任务现在跑到哪、出了啥结果。问题是它一关机或重启,脑子里没存盘的就全没了,像个临时记录员。

易失/会丢:指引擎记在'脑子里'(内存)的东西没落到硬盘,断电或重启就消失。所以才要赶紧抄一份到能长期保存的地方。

存档/数据库:把运行记录长期保存下来的那本'正式账本',断电也不丢。监控查历史、引擎挂了时都靠它。

就地随手存(穿透写):不另派人事后抄账,而是在引擎干活的当口直接顺手把账记下来,任务一结束那一刻账就落地了,不用等下一轮抄。

钩子(挂点):引擎特意留出来的几个'插口',让你能在'任务开始/完成/出错/发消息'这几个时刻插进自己的动作。本方案就是在这几个插口上随手存账。

壹 解决什么技术问题 *

AI 给出的判断(比如"这个候选人不合格")到底有没有依据?用同一家的 AI 去检查它自己,它会偏袒自己的结论、查不出问题;检查用的 AI 自己也会随版本"飘",还可能因为没钱 / 故障中断,反而拖累正经业务。

贰 用什么技术手段 *

- 跨厂商组陪审团。参与核查的几个 AI 必须来自彼此不同的厂商、且都不能和生产用的是同一家,从根上堵住"自己人偏袒自己人"。
- 有分歧才投票。先让主评委判,陪审一致就直接采纳;只有主评委和陪审吵起来才召集投票、谁多数听谁的,既省钱又只在争议处加码。
- 核查 AI 挂了不影响主业务。核查用的 AI 一旦不可用,就把这条当作没查、直接跳过,绝不报错,真正权威的规则判定照常运行。
- 拿人工抽检当标尺。让人工标好正确答案,跟评委判断逐条对账,算出评委的准确率,知道它到底值不值得信。
- 分清"改了规则"还是"AI 飘了"。给评分标准打版本号,只拿同版本前后两次判断比一致率,把人为调整和模型自身漂移区分开,不误报。

叁 达到什么技术效果 *

- ✓ 核查更客观:不同厂商互查,避免 AI 偏袒自己。
- ✓ 成本可控、可复现:同一条记录最多只花一次钱核查,重跑结果一致。
- ✓ 核查环节出故障,绝不连累真正的业务判定。
- ✓ 能用数字说清"这个 AI 评委到底有多可信",失准就调参或停用。

建议标题 （可选 · 供命名参考）

一种用多家 AI 组成"陪审团"持续核查 AI 业务结论是否有依据的方法及系统

补充说明 （可选）

应用场景。对招聘等业务里 AI 给出的判定(如"候选人是否合格")做持续的事实核查,确保结论真有依据。

与常规做法的差异。常规做法只请一个 AI 来给答案打分,而且常常跟生产用的是同一家 AI,等于让人给自己的卷子打分,容易偏袒。本方案有四处别人不容易绕过的设计拧成一股绳:一是用一套死规矩硬性挑出几家不同厂商的 AI 当陪审,谁跟生产同厂就不许进,杜绝偏袒;二是主评委和陪审一致就直接定,不一致才召集投票,既省钱又只在吵架处加码;三是核查 AI 挂了就当这条没查、绝不报错也不拖累正经业务;四是给评分标准打了版本号,从而能分清是人改了规则还是 AI 自己飘了。少一环都达不到这个又准又稳又省的效果。

名词解释。

结论有没有依据(接地性):AI 给的判断是不是真有规则和简历、岗位这些事实撑着,而不是张口就来。撑得越足分越高。

陪审团:一组当评委的 AI,像法庭陪审一样集体判一个结论靠不靠谱,不让一家说了算。

主评委:先上场打分的那位评委 AI;它和陪审一致就直接定,意见不合才召集大家投票。

不同厂商的 AI(模型族):把各家大模型按娘家分门别类,比如这家那家;同一家的算自己人,容易互相偏袒,所以评委要挑别家的。

评委挂了就跳过(故障旁路):核查用的 AI 没钱了或出故障,就把这条当作没查、直接略过,绝不报错也不拖累正经业务。